

Guida a SSH

Connessioni sicure, chiavi, config e port forwarding

SSH (Secure Shell) è il modo standard per collegarti a macchine remote in modo cifrato: login shell, esecuzione comandi, copia file e tunnel di rete.

Quick reference

Comando	Descrizione
<code>ssh user@host</code>	Connessione base
<code>ssh -p 2222 user@host</code>	Connessione su porta diversa
<code>ssh -i ~/.ssh/id_ed25519 user@host</code>	Specifica chiave
<code>ssh -v user@host</code>	Debug connessione (verbose)
<code>ssh-keygen -t ed25519</code>	Genera coppia di chiavi
<code>ssh-add ~/.ssh/id_ed25519</code>	Carica chiave nell'agent
<code>scp file user@host:/path</code>	Copia file su remoto
<code>ssh -L 8080:localhost:80 user@host</code>	Tunnel locale (port forward)
<code>ssh -J bastion user@target</code>	Jump host / bastion

Indice

1	Cos'è SSH	3
2	Concetti fondamentali	3
3	Primo utilizzo	3
3.1	Connessione base	3
3.2	Porta diversa	4
3.3	Usare una chiave specifica	4
4	Chiavi SSH (setup consigliato)	4
4.1	Generare una chiave moderna (ed25519)	4
4.2	Caricare la chiave nell'agent	4

5	Aggiungere la chiave al server	4
5.1	Metodo 1: ssh-copy-id (se disponibile)	4
5.2	Metodo 2: senza ssh-copy-id (compatibile ovunque)	5
6	Configurare ~/.ssh/config	5
7	Copia file: scp e rsync	5
7.1	scp (semplice)	5
7.2	rsync (piu efficiente su cartelle grandi)	5
8	Port forwarding (tunnel)	5
8.1	Tunnel locale (-L)	6
8.2	Tunnel remoto (-R)	6
8.3	SOCKS proxy (-D)	6
9	Jump host (bastion)	6
9.1	Usare -J	6
9.2	Configurazione con ProxyJump	6
10	Troubleshooting veloce	6
10.1	Debug con verbose	7
10.2	Errori tipici	7
10.3	Permessi della cartella .ssh	7
11	Cheat sheet veloce	7
12	Errori comuni dei principianti	8
13	Conclusione	8

1 Cos'è SSH

SSH è un protocollo che ti permette di stabilire una connessione cifrata tra un client (il tuo computer) e un server remoto.

In pratica lo userai per:

- entrare su un server e lavorare da shell;
- eseguire comandi remoti;
- trasferire file (`scp`, `sftp`, `rsync -e ssh`);
- creare tunnel di rete (port forwarding) per accedere a servizi non esposti.

Idea pratica: se impari bene SSH, diventa la "colla" tra sviluppo locale, server e strumenti (Git, deploy, DB, dashboard interne).

2 Concetti fondamentali

Client / Server

Il client è il comando `ssh` sul tuo computer. Il server è il demone `sshd` sulla macchina remota.

Porta

Di default SSH usa la porta 22, ma spesso viene cambiata (es. 2222).

Chiave pubblica / privata

Autenticazione moderna: la chiave privata resta sul client; la pubblica viene messa sul server in `~/.ssh/authorized_keys`.

known_hosts

Il file in cui il client salva le impronte dei server visitati per evitare attacchi man-in-the-middle.

Struttura mentale:

Client → Server → Autenticazione (password o chiavi)

3 Primo utilizzo

3.1 Connessione base

```
ssh user@host
```

Dove:

- `user` è l'utente sul server (es. `ubuntu`, `root`, `deploy`);
- `host` è un nome DNS o un IP.

3.2 Porta diversa

```
ssh -p 2222 user@host
```

3.3 Usare una chiave specifica

```
ssh -i ~/.ssh/id_ed25519 user@host
```

Consiglio: se stai usando chiavi, punta a non passare `-i` ogni volta: metti la configurazione in `~/.ssh/config`.

4 Chiavi SSH (setup consigliato)

4.1 Generare una chiave moderna (ed25519)

```
ssh-keygen -t ed25519 -C "nome@azienda.com"
```

Di default troverai:

- chiave privata: `~/.ssh/id_ed25519`
- chiave pubblica: `~/.ssh/id_ed25519.pub`

Passphrase: mettila. È una protezione importante se la chiave privata viene copiata o rubata.

4.2 Caricare la chiave nell'agent

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

Su macOS spesso è utile anche salvare la chiave nel portachiavi (opzionale):

```
ssh-add -apple-use-keychain ~/.ssh/id_ed25519
```

5 Aggiungere la chiave al server

5.1 Metodo 1: ssh-copy-id (se disponibile)

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host
```

5.2 Metodo 2: senza ssh-copy-id (compatibile ovunque)

```
cat ~/.ssh/id_ed25519.pub | ssh user@host "mkdir -p ~/.ssh && cat »  
~/.ssh/authorized_keys"
```

Check veloce: dopo aver aggiunto la chiave, riprova `ssh user@host`. Se tutto è ok, non ti chiederà la password (ma potrebbe chiedere la passphrase della chiave).

6 Configurare ~/.ssh/config

Con `~/.ssh/config` puoi creare alias e standardizzare porte, user e chiavi.

Esempio:

```
Host myserver  
HostName 203.0.113.10  
User ubuntu  
Port 2222  
IdentityFile ~/.ssh/id_ed25519  
IdentitiesOnly yes
```

Da quel momento:

```
ssh myserver
```

IdentitiesOnly yes evita tentativi con troppe chiavi (errore tipico: `Too many authentication failures`).

7 Copia file: scp e rsync

7.1 scp (semplice)

```
scp file.txt user@host:/tmp/  
scp -r cartella/ user@host:/var/www/
```

7.2 rsync (più efficiente su cartelle grandi)

```
rsync -av -progress -e ssh cartella/ user@host:/var/www/
```

8 Port forwarding (tunnel)

Il port forwarding serve per "portare" un servizio remoto in locale (o viceversa) senza esporlo pubblicamente.

8.1 Tunnel locale (-L)

Esempio: hai un servizio sul server su `localhost:80` e vuoi vederlo in locale su `localhost:8080`.

```
ssh -L 8080:localhost:80 user@host
```

8.2 Tunnel remoto (-R)

Esempio: vuoi esporre una porta locale (es. 3000) sul server remoto.

```
ssh -R 3000:localhost:3000 user@host
```

8.3 SOCKS proxy (-D)

Crea un proxy SOCKS locale (utile per navigare tramite il server):

```
ssh -D 1080 user@host
```

Consiglio: aggiungi `-N` quando ti serve solo il tunnel (senza aprire shell).

9 Jump host (bastion)

Quando il server target non è direttamente raggiungibile, passi da un bastion.

9.1 Usare -J

```
ssh -J user@bastion user@target
```

9.2 Configurazione con ProxyJump

```
Host bastion
HostName 198.51.100.10
User ubuntu
Host target
HostName 10.0.1.25
User ubuntu
ProxyJump bastion
```

10 Troubleshooting veloce

10.1 Debug con verbose

```
ssh -v user@host
ssh -vvv user@host
```

10.2 Errori tipici

Permission denied (publickey)

La chiave non e sul server, oppure stai usando la chiave sbagliata.

Host key verification failed

Il server e cambiato (o la chiave in `known_hosts` non coincide). Verifica prima di rimuovere l'entry.

Connection refused / timed out

Porta sbagliata, firewall, o `sshd` non in ascolto.

10.3 Permessi della cartella `.ssh`

Su molti server, permessi troppo aperti bloccano l'autenticazione a chiave.

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

11 Cheat sheet veloce

Comando	Significato
<code>ssh user@host</code>	Login remoto
<code>ssh -p 2222 user@host</code>	Porta custom
<code>ssh -i key user@host</code>	Specifica chiave
<code>ssh -v user@host</code>	Verbose (debug)
<code>ssh-keygen -t ed25519</code>	Genera chiave
<code>ssh-add key</code>	Carica chiave nell'agent
<code>scp file user@host:/path</code>	Copia file
<code>rsync -e ssh src/ user@host:dst/</code>	Sync efficiente
<code>ssh -L lport:host:rport user@host</code>	Tunnel locale
<code>ssh -R rport:host:lport user@host</code>	Tunnel remoto
<code>ssh -D 1080 user@host</code>	SOCKS proxy
<code>ssh -J bastion user@target</code>	Jump host

12 Errori comuni dei principianti

1. Riutilizzare la stessa chiave ovunque

Meglio chiavi separate per contesti diversi (lavoro, personale, produzione).

2. Usare root direttamente

Preferisci un utente normale + `sudo` (quando serve).

3. Disabilitare i controlli di sicurezza a caso

Evitare di forzare `StrictHostKeyChecking no` senza capire cosa implica.

13 Conclusione

SSH è uno strumento semplice all'apparenza, ma potentissimo quando lo usi bene: chiavi + config + tunnel.

Da ricordare:

- usa chiavi `ed25519` con passphrase;
- centralizza alias e opzioni in `~/.ssh/config`;
- usa i tunnel per accedere a servizi interni senza esporli;
- se qualcosa non funziona: `ssh -v` e controlla permessi.